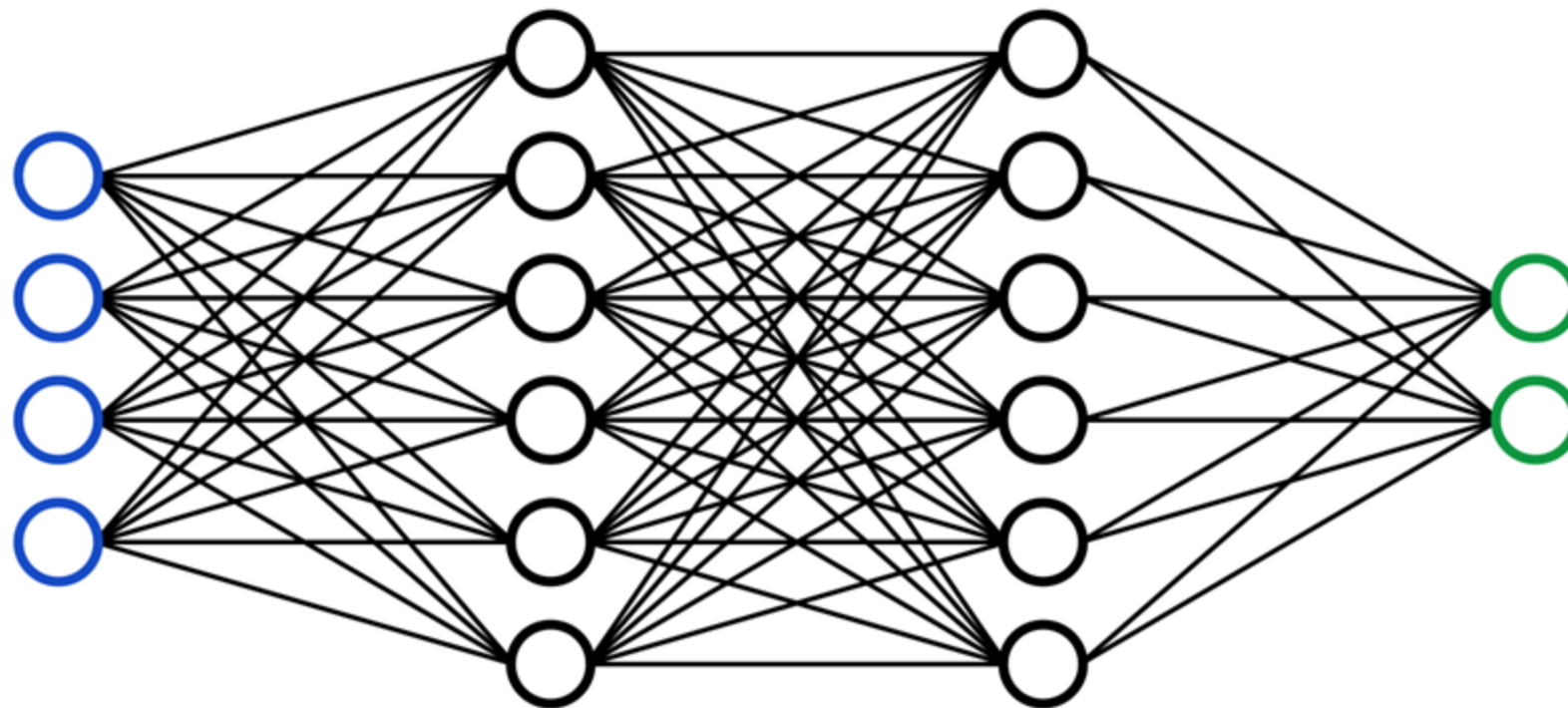




Deep Learning

Presented to CS Club Symposium, July 18, 2026

Jason Miller, PhD

Professor of Computer Science at Hood College

Topics

- Artificial Intelligence (AI)
 - Expert Systems vs Machine Learning (ML)
- Deep Learning with Neural Network (NN) models
 - Neuron, activation, multi-layer Perceptron
- Training Regimes
 - Loss function, gradient descent, validation, overfitting
- Transformers and Large Language Models (LLM)
 - Embedding layers, Attention layers, and other extensions
- MNIST tutorial: Try it at Home

Branches of Artificial Intelligence (AI)

- Expert Systems

- Simple version: program in the rules

- Example: Python program that says, “if spot on skin, test for melanoma”

- Complex version: knowledge base + symbolic logic engine

- Example: Prolog program that reads “melanoma is cancer” and “cancer is a disease”, then deduces that “melanoma is a disease”

- Machine Learning

- Design a mathematical model with parameters (variables or weights)

- Adjust the parameters to match the available (training) data

- Example: make a model with a weight for every possible (symptom | disease) pair.
 - Review medical records (training data).
 - Up-weight common associations, like (skin spot | melanoma).
 - Down-weight uncommon associations, like (sneezing | melanoma).

Branches of Artificial Intelligence (AI)

AI

Expert Systems

Machine Learning

Neural Networks

- Perceptrons (feed-forward)
- Convolutional NNs
- Language Models

Other Types

- Linear Regression
- Support Vector Machine
- Random Forest
- Gradient Boosting Machine

Pioneers in Artificial Intelligence (AI)



Alan Turing

We need computers that learn the way children do. (1950)

Challenge, part 1, AI

- Name two branches of Artificial Intelligence (AI).
- What's the difference?

Machine Learning: Linear Regression

$$\mathbf{y} = f(\mathbf{x})$$

Assume you can predict $\mathbf{y}$ given $\mathbf{x}$.

Say $\mathbf{y}$ is a function of $\mathbf{x}$, and f is the function.

$$\mathbf{y} = \mathbf{m}\mathbf{x} + \mathbf{b}$$

Build a model. This model has 2 parameters: $\mathbf{m}$ and $\mathbf{b}$.
This model assumes a linear relationship.

$$y = 3x + 2$$

Train the model. Review your training data.

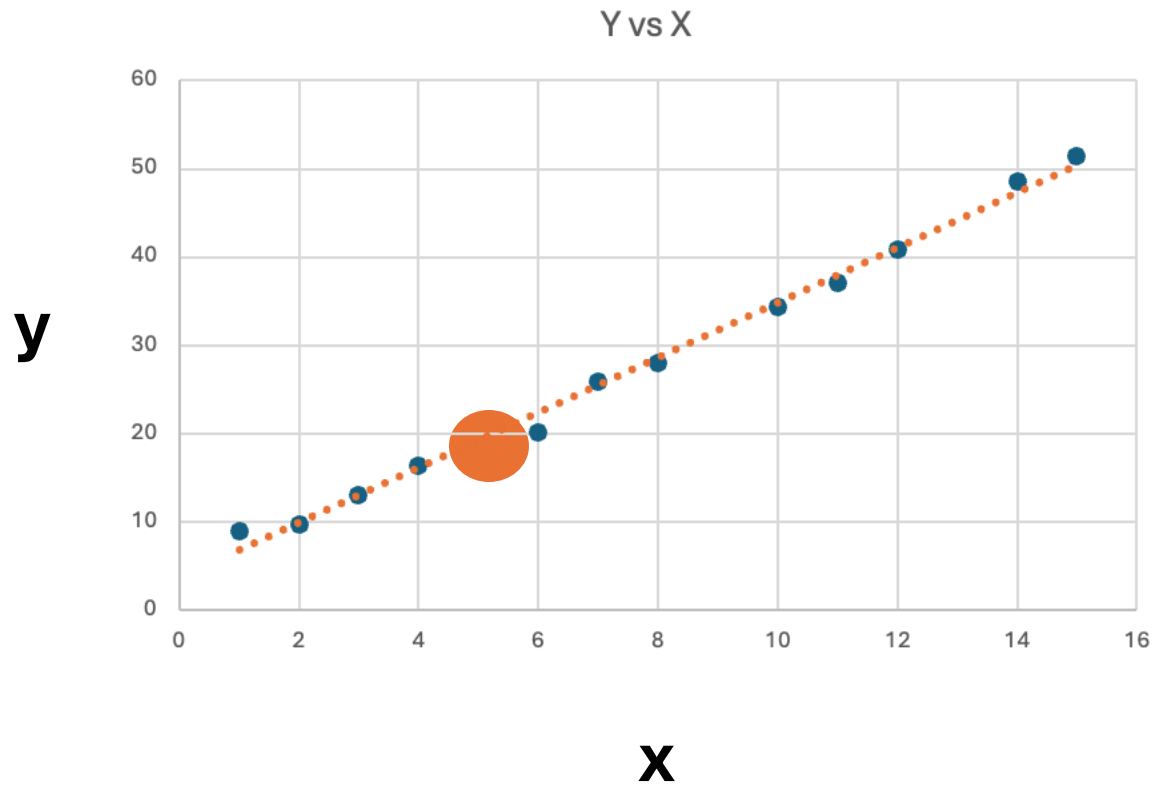
Learn that $\mathbf{m}=3$ and $\mathbf{b}=2$ works well in most cases.

$$17 = 3 * 5 + 2$$

Deploy the trained model, make predictions.

Here, we predict that $y=17$ if $x=5$.

Machine Learning: Linear Regression



Initial model: $y = mx + b$

● Training data

..... Trained model: $y = 3x + 2$

● Prediction: $y = 17$ if $x = 5$

Pioneers in Artificial Intelligence (AI)



Karl Friedrich Gauss

Invented Linear Regression in 1800.
Used it extensively. Never published it.
May have thought it was obvious.

Challenge, part 2, ML

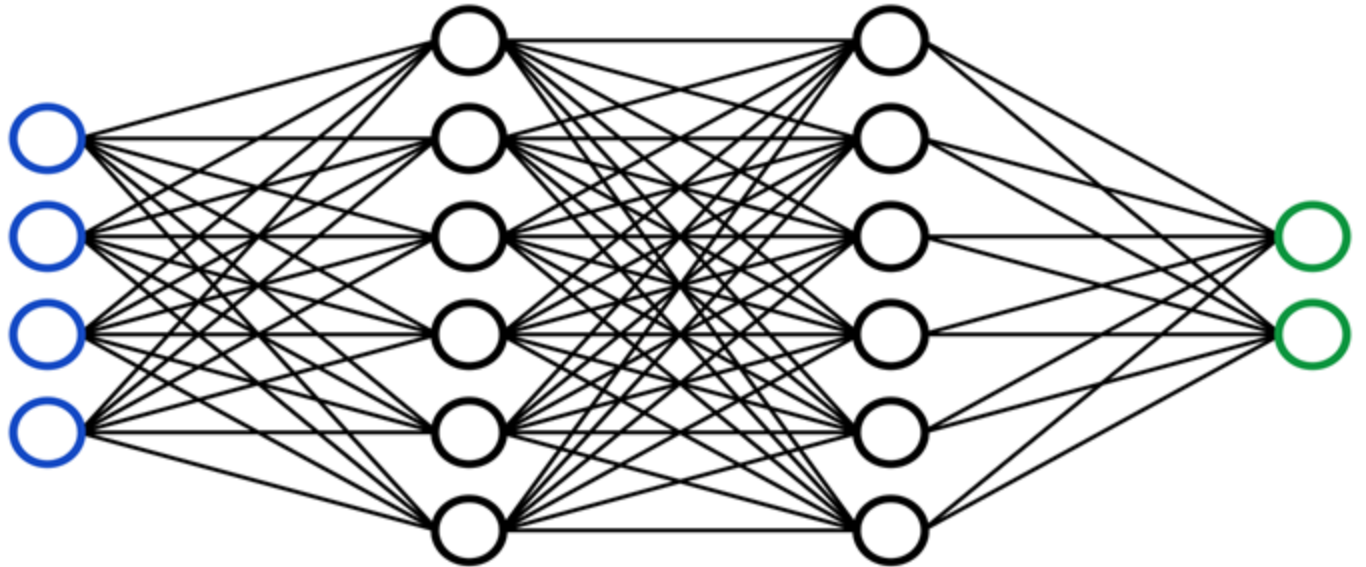
- What is a parameterized model?
- How do we set the parameter values of a model?
- Are today's Large Language Models (LLMs) examples of parameterized models?

Machine Learning: Neural Network

$\mathbf{y} = f(\mathbf{X})$ **Make an assumption:** we can predict $\mathbf{y}$ from $\mathbf{x}$.

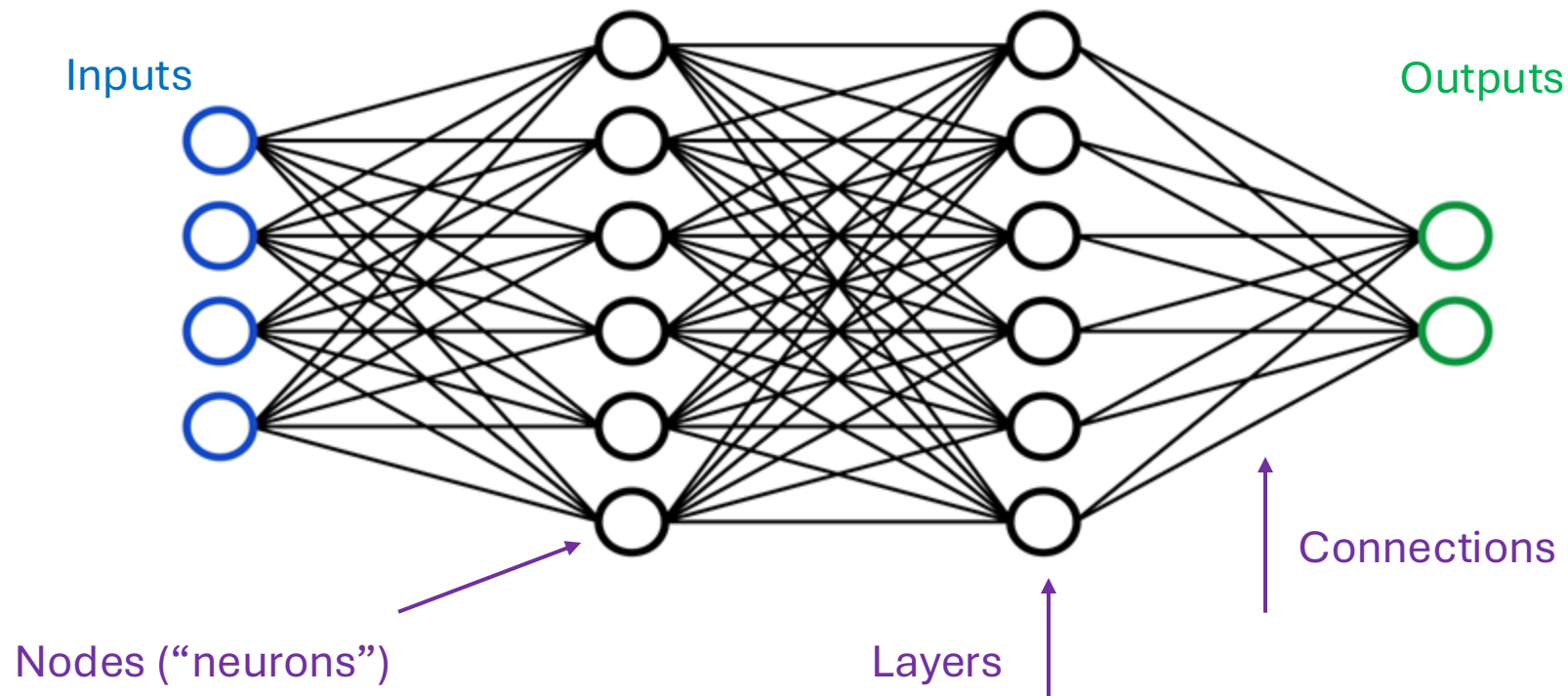
Choose a model: a **neural network** is a great choice!

- Flexible
- Scalable
- Easy to train
- Easy to deploy
- Lots of literature
- Many tutorials
- Open-source code



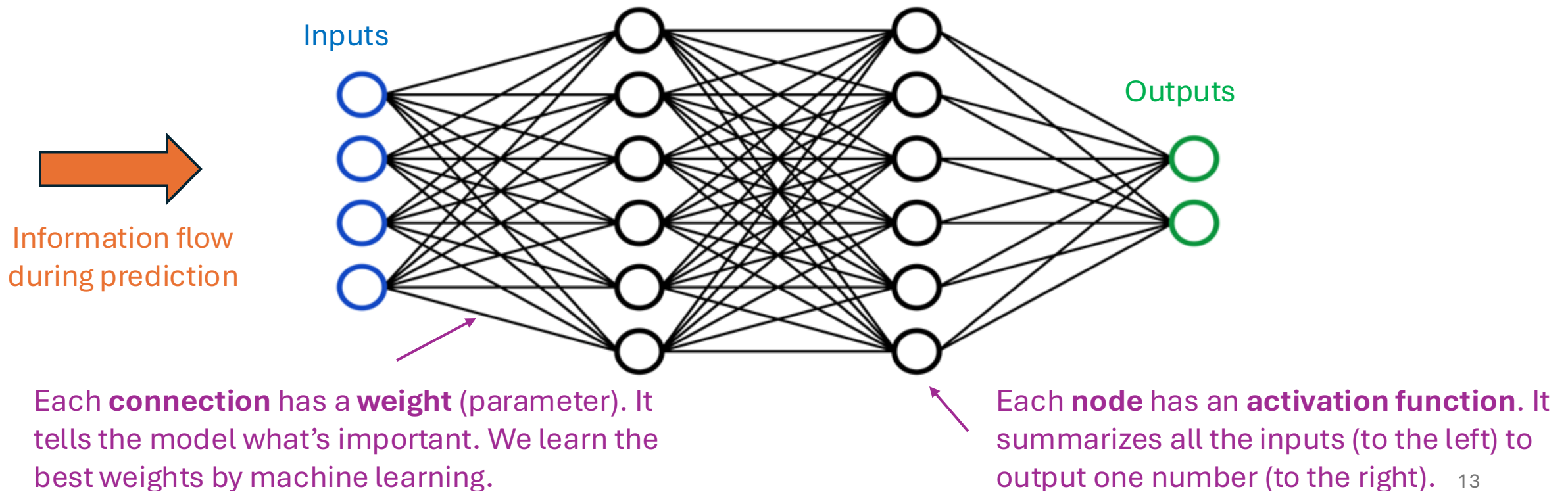
Neural Network: components

This neural network design is called **feed-forward** or multi-layer perceptron (**MLP**).



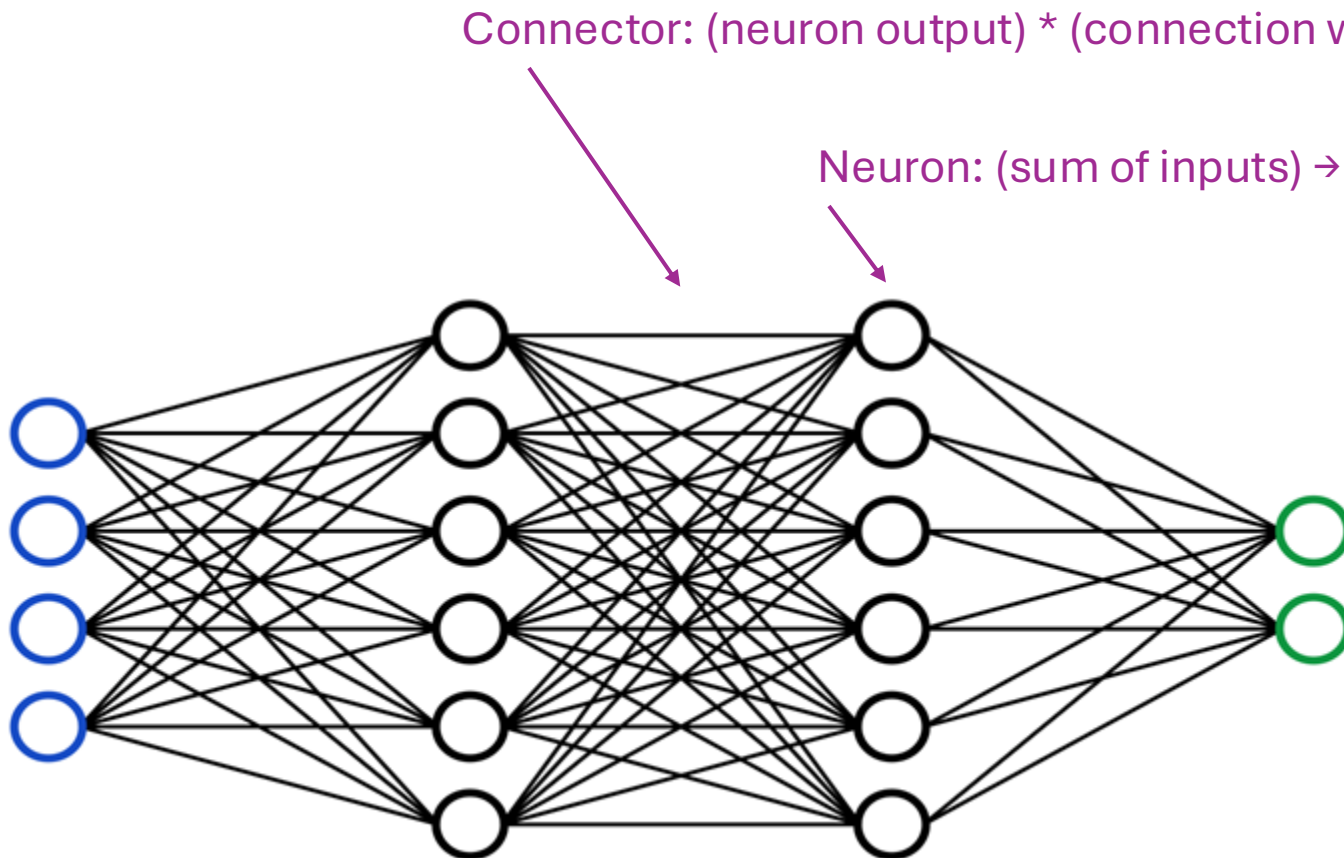
Neural Network: information flow

Nodes are organized in **layers**. Each layer is **fully connected** to the previous.



Neural Network: inside a neuron

The “**layers of neurons**” design emulates the human brain.



You get to choose the activation function. The best ones are non-linear and differentiable.

A simple example is **ReLU(x)**:

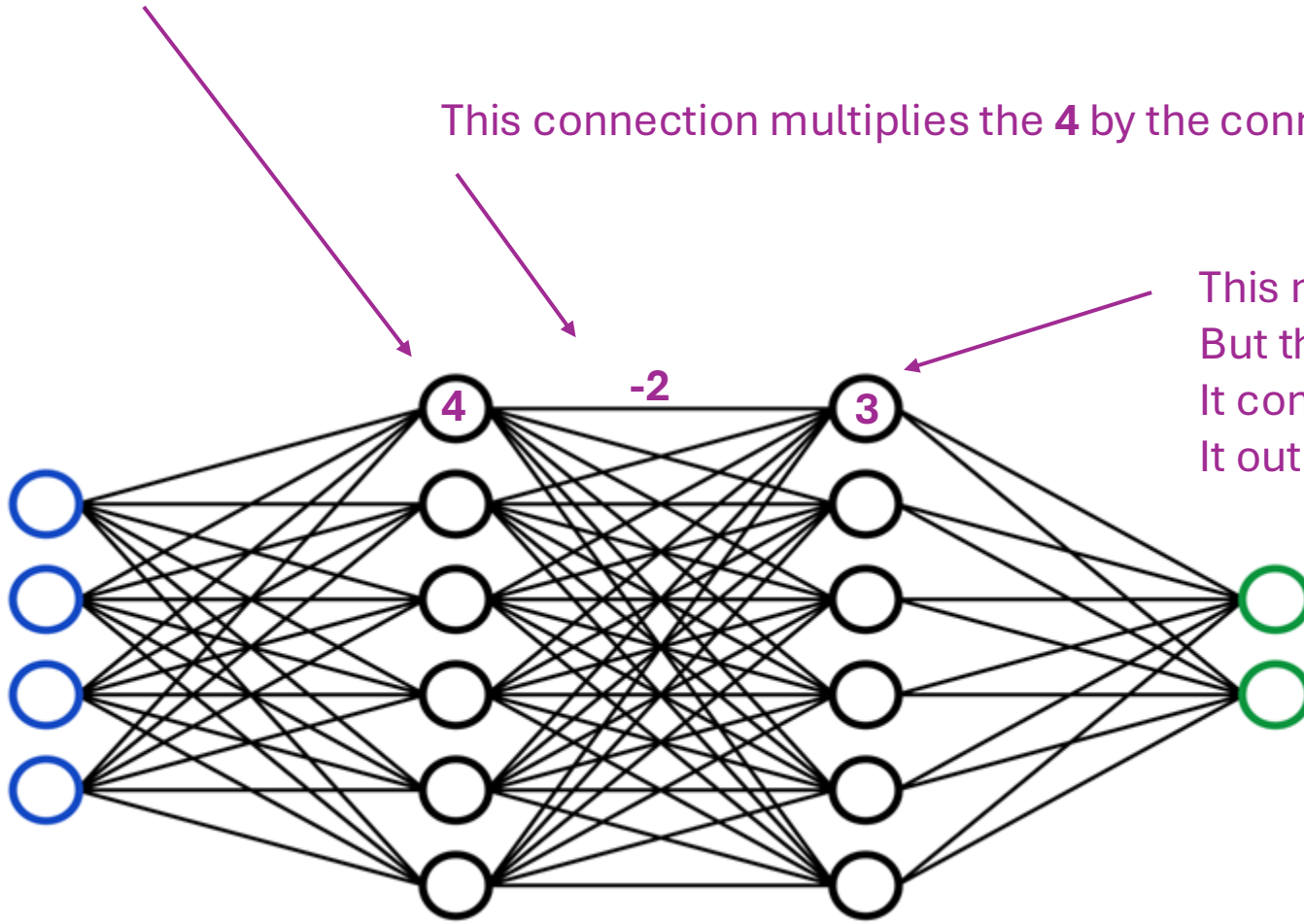
- Input **x**
- If **x** is negative, output **zero**
- Else, output **x**

Neural Network: example

If this neuron computes a **4**, it sends **4** through each downstream connector.

This connection multiplies the **4** by the connector weight, which could be **-2**.

This neuron receives **-8** as one input.
But the sum of all its inputs is perhaps **3**.
It computes **ReLU(3)** which is **3**.
It outputs **3** through each downstream connector.

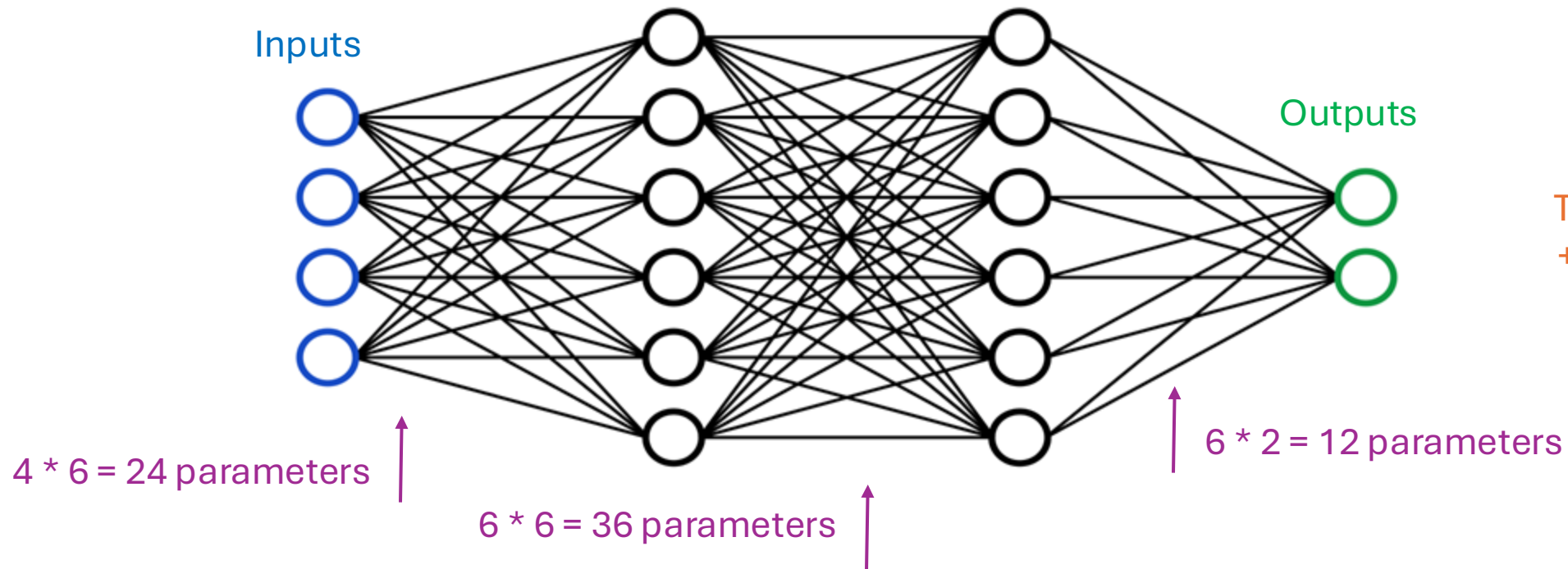


Neural Network: complexity

How many **nodes**? How many **layers**? That is up to you!

Each **connection** has a **weight** (a trainable parameter).

The more connections, the more parameters you need to train.



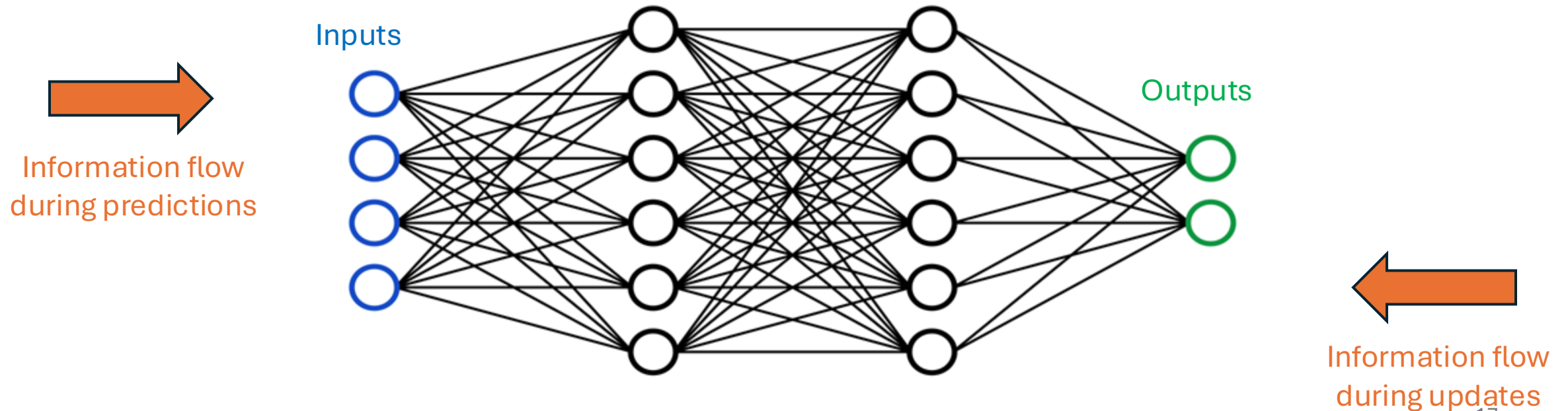
This *very small* MLP has $24 + 36 + 12 = 72$ parameters.

Neural Network: training basics

Initialize weights to random numbers.

Repeat

1. Forward compute: let the model make predictions on training data.
2. Score the results: good or bad.
3. Backward compute: update all the weights to do better next time.



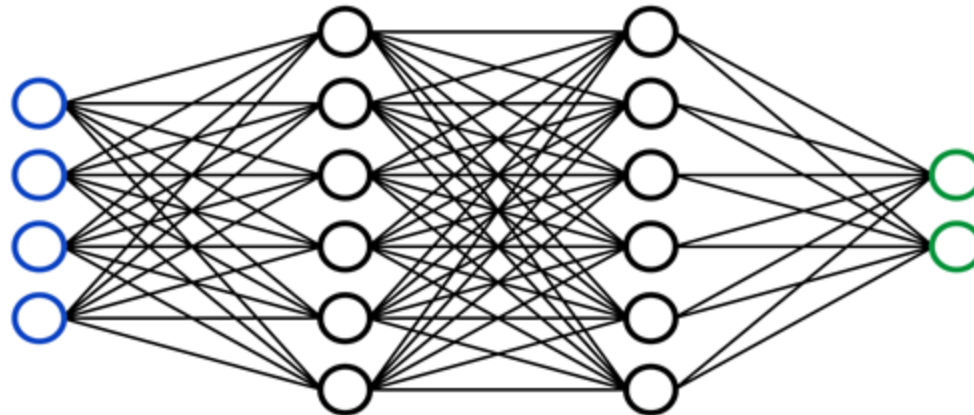
Neural Network: hyperparameters

Parameters: you **learn** the values

- Every connection weight is a trainable parameter.

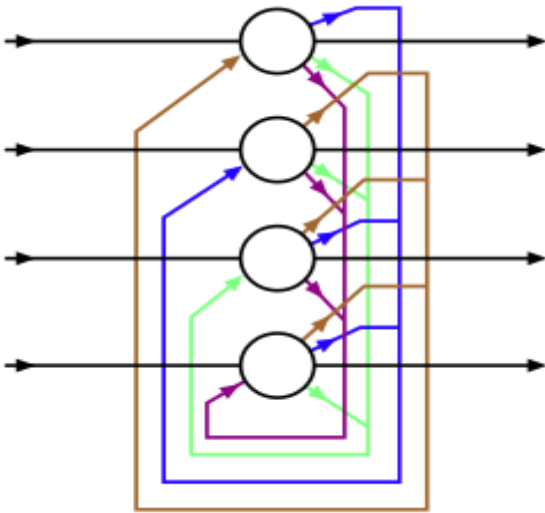
HyperParameters: you **choose** the values

- Number of layers. Number of neurons per layer.
- Activation functions. Regularizations. Learning rate. Overall loss function.
- Size of training set. Size of validation set. Number of cross validation rounds.
- Stopping criteria or number of epochs per trained model.
- Choosing good values is an art, not a science.



Pioneers in Artificial Intelligence (AI)

Nobel Prize in Physics, 2024



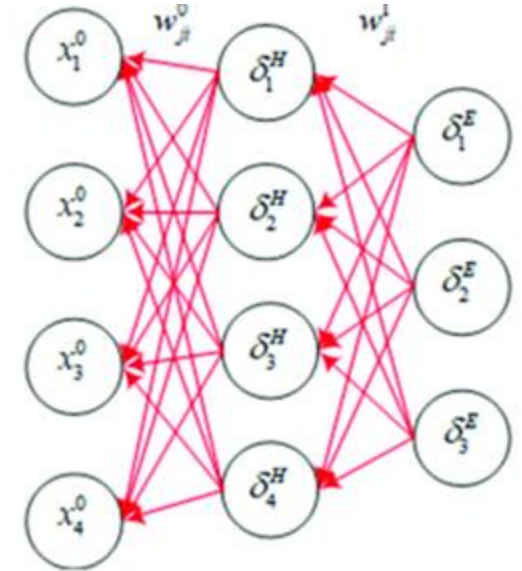
John Hopfield

Built neural networks to understand human associative memory. **Hopfield Networks** are used in many electronic devices.



Geoffrey Hinton

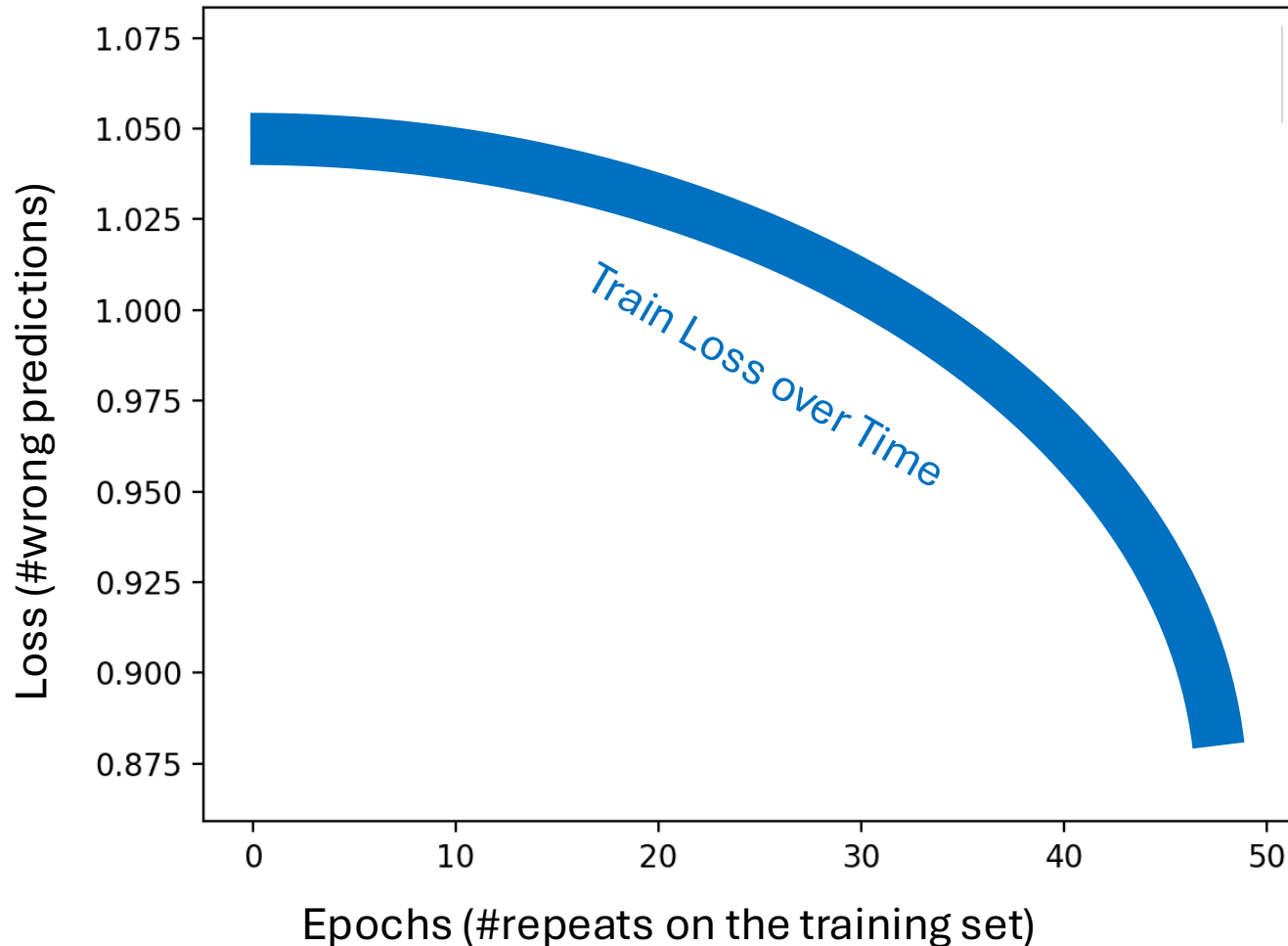
Developed the “**backward** computation” that allows neural networks to learn from their mistakes. Created **AlexNet** (2012), the first great image classifier.



Challenge, part 3, NN

- What is the biological inspiration for building neural networks (NNs) with computers?
- How are neurons connected within a multi-layer Perceptron (MLP)? Hint: MLP is also called feed-forward, fully-connected.

Machine Learning: training



Choose a loss function

- Simple: number of wrong predictions
- Better: distance from truth
- Advanced: add in mathematical penalties to avoid various pitfalls (“regularization”)

Evaluate, Update, Repeat

- For each batch of data...
- Make predictions, calculate the loss
- Update weights to reduce the loss

The alignment problem

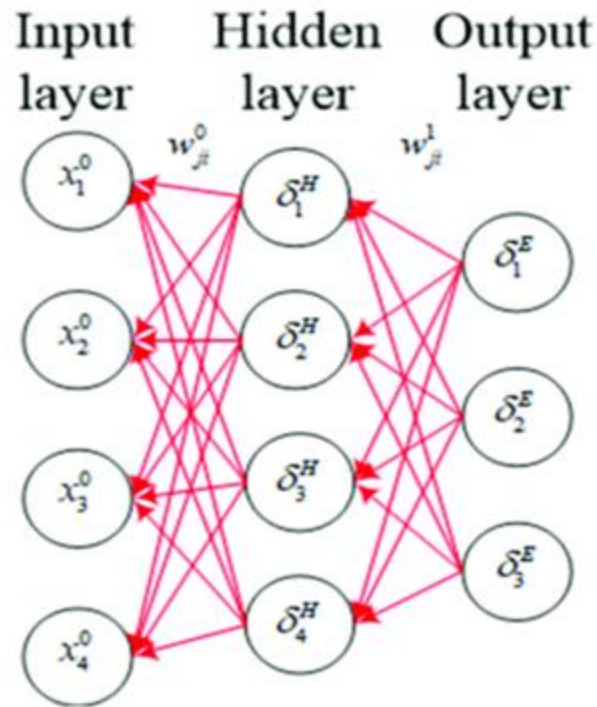
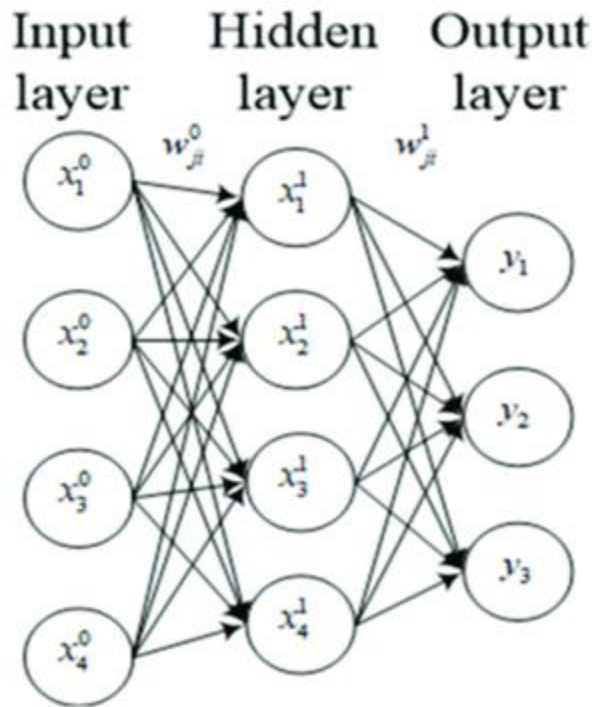
- Training reduces loss, that’s all
- Critical to choose a loss function that aligns with your real-world goals

Training means *reduce-the-loss*

We use **labeled** training data *i.e.* we know the right answers.
In a loop: we predict, measure loss, update weights.



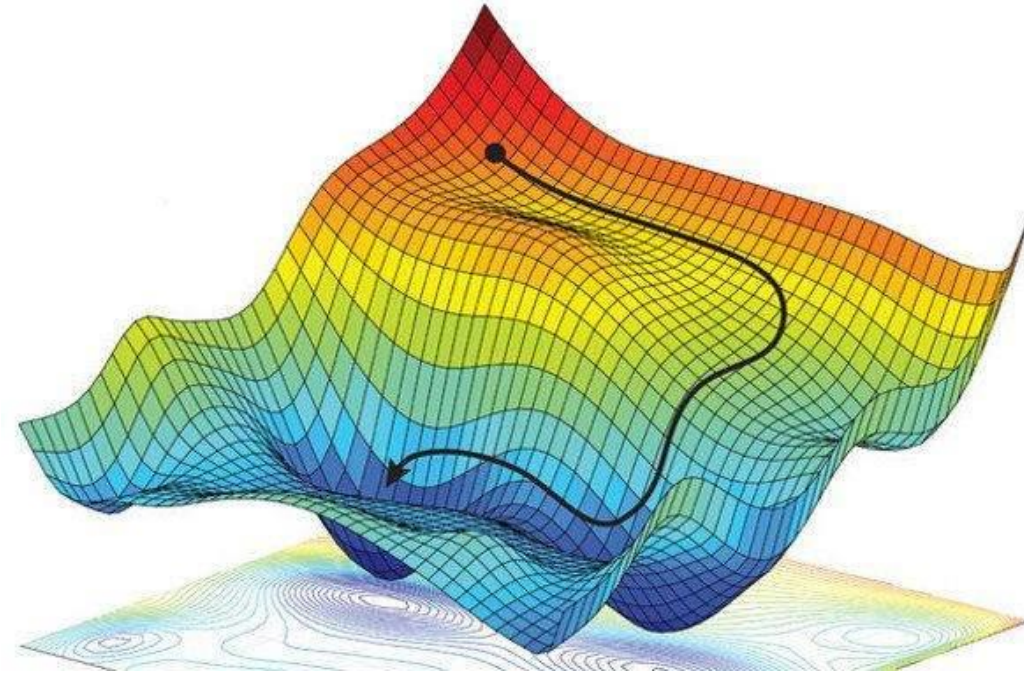
Left to right: input training samples, make a predictions, compute the loss.



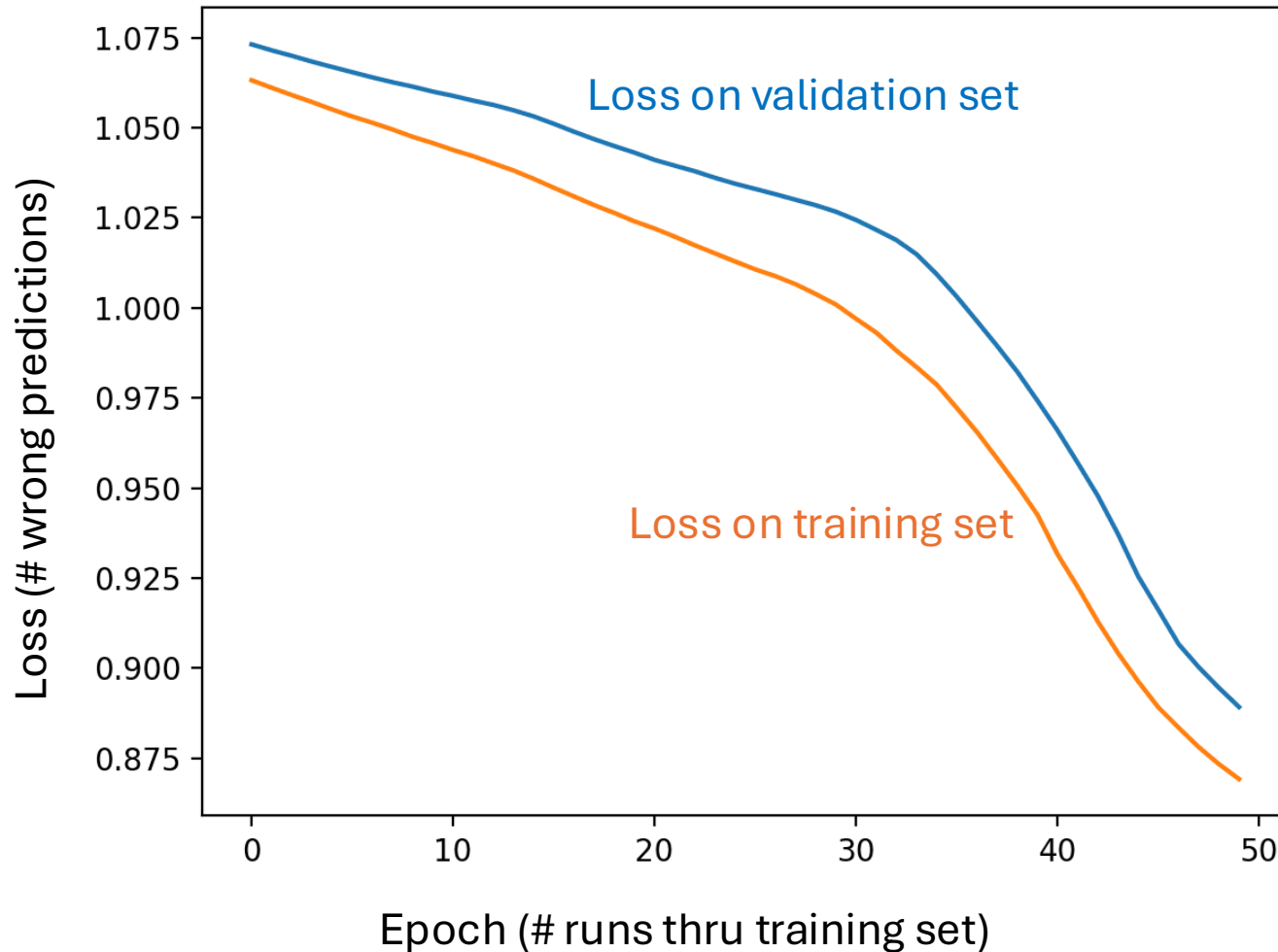
Right to left: increase and decrease weights so as to reduce loss. *Rely on partial derivatives of the loss.*

Training means *reduce-the-loss*

- “Stochastic Gradient Descent (SGD)”
- Start with a random guess
 - Initialize weights randomly
- Repeat till satisfied:
 - Evaluate on a batch of training data
 - Update the weights to reduce loss
- Visualization
 - Two parameters on X & Y axis
 - Loss on vertical Z axis (lower is better)
 - Black arrow shows loss over time



Validate while you Train



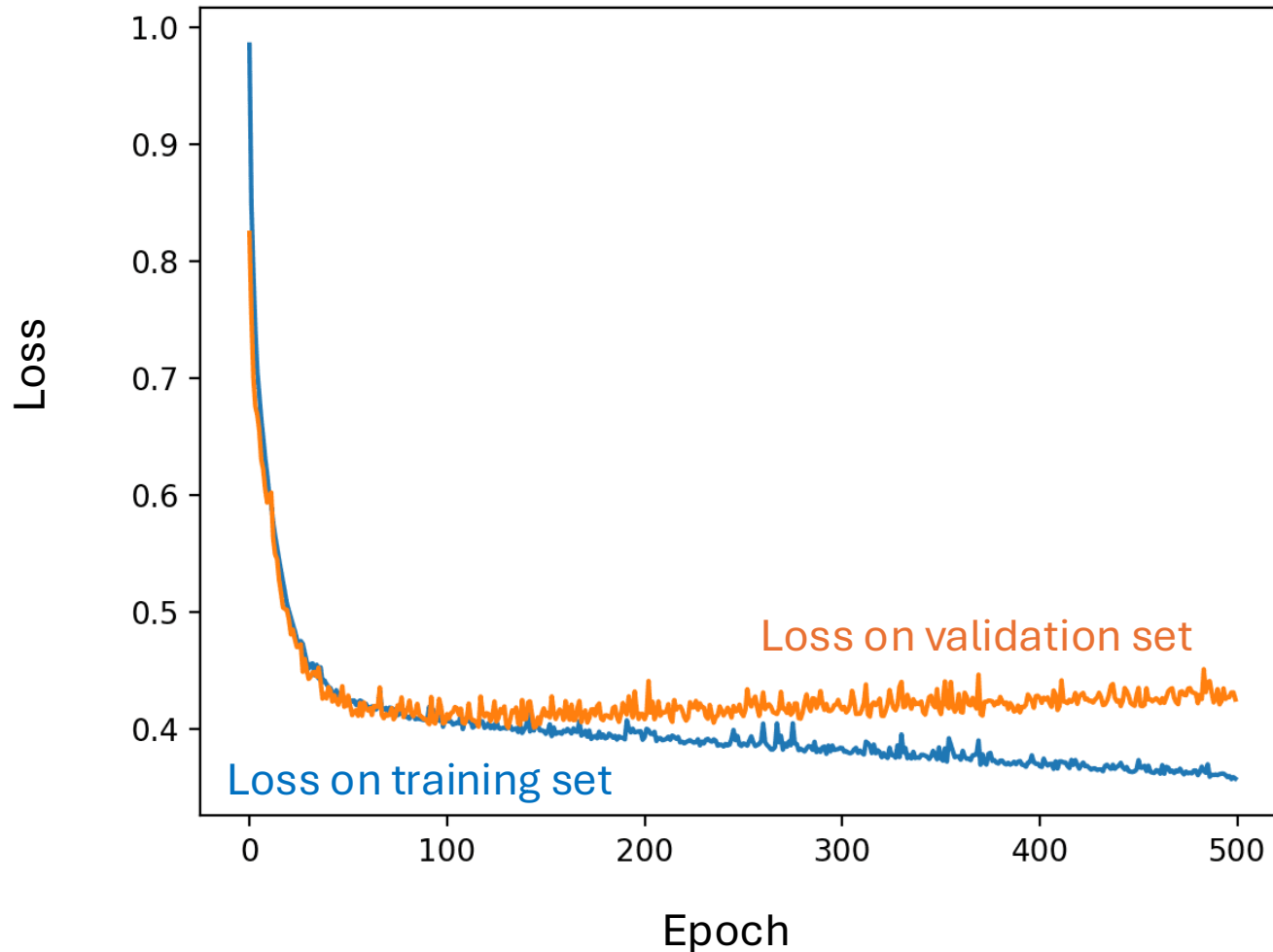
Training Loss – trustworthy?

- Loss measured on training samples
- Same samples the model was trained on
- This is circular reasoning!

Validation Loss – more trusted

- A random subset of the training samples
- Not used to update model weights
- Used only to evaluate performance
- The situation shown here is common: worse performance on the validation set. Why?

Avoid Overfitting



Problem

- Training loss is going down (*good!*)
- Validation loss is going up (*bad!*)
- Divergence began at about epoch 50

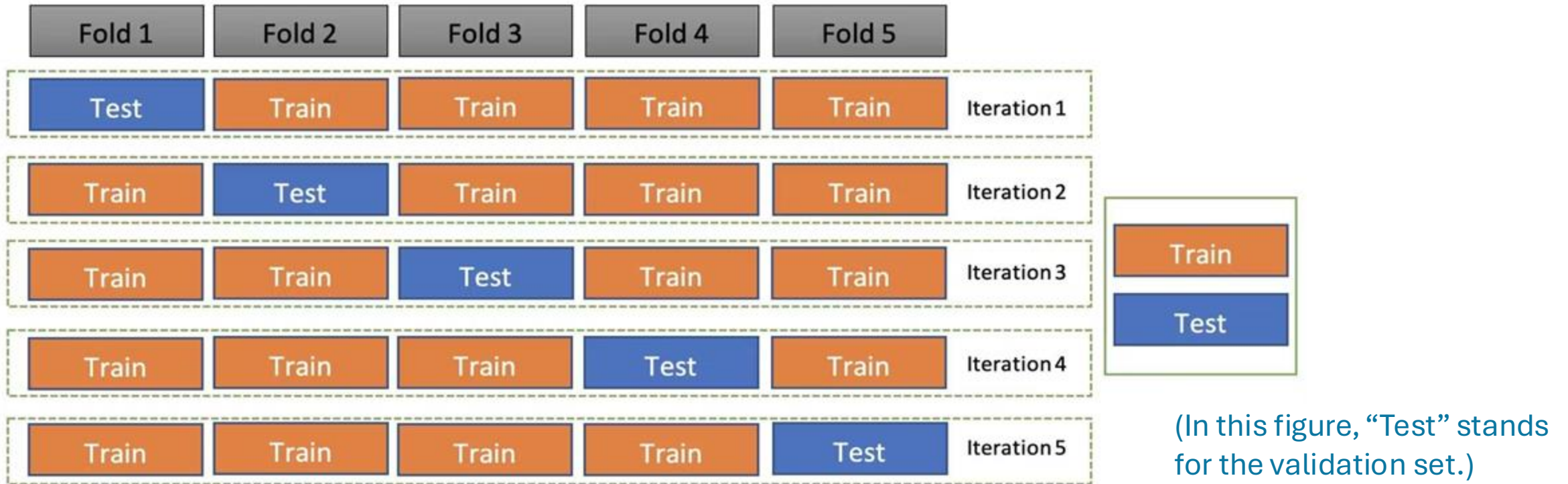
Explanation

- Model is memorizing the training set
- **Model is too smart for its own good!**

Possible Solutions

- Stop the training at the first sign of overfitting (“early stopping”)
- Use a simpler model (“not as smart”)
- Add a **regularizer** – a mathematical technique to slow down learning

Technique for Train and Validate



In Five-Fold Cross-Validation (**5FCV**), you train 5 models in preparation. You train each model on 4/5 of the training data and validate it on the remainder. If the validation performance is consistently good, then retrain one model on the full training set.

Training tips*

For choosing random weights, use an **initializer**.

For updating weights, use an **optimizer**.

Set aside a random portion of the training data for **validation** purposes.

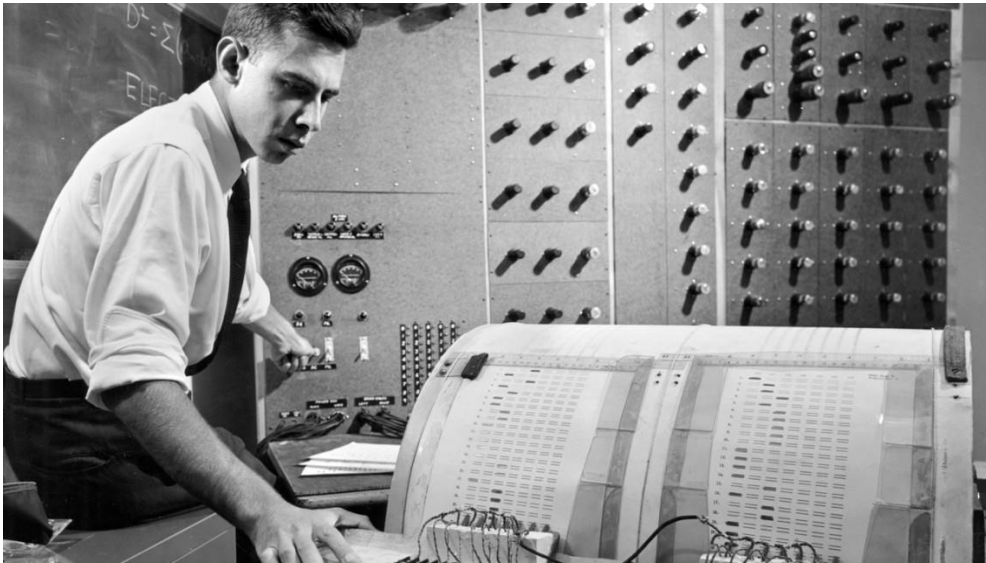
To exploit parallelism, process training sets in small **batches**.

If feasible, train on all the data repeatedly (each round is called an “**epoch**”).

Randomize the sample order before each epoch.

*These are implemented by default in coding libraries like Keras, TensorFlow, and PyTorch.

Pioneers in Artificial Intelligence (AI)



Frank Rosenblatt

His is **Perceptron** was trained - not programmed - to distinguish big letters written on cards. The machine was made of 400 cameras and 520 vacuum tubes.

Challenge, part 4, training

- What does it mean to run one epoch of training?
- Why should you validate while training a model?

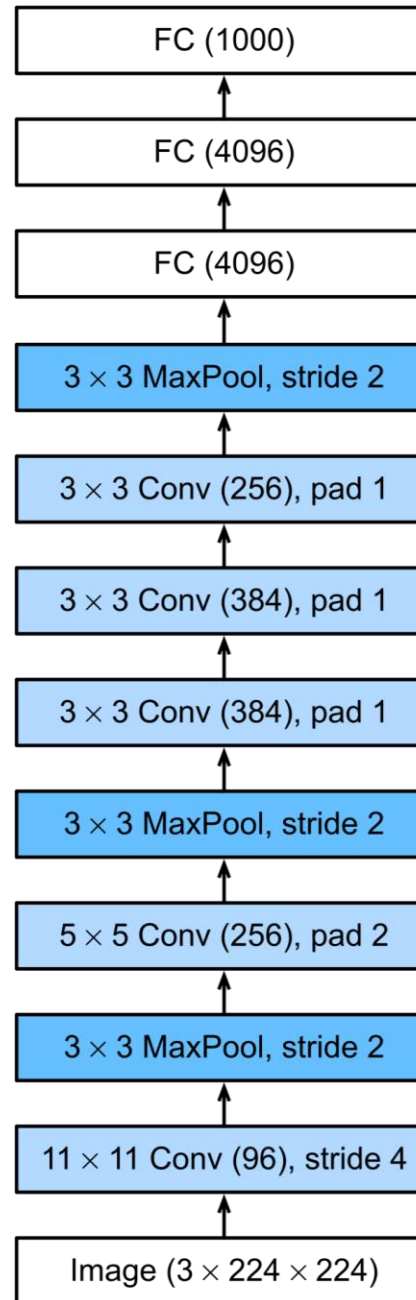
Neural Network extensions

- Basic neural network: the multi-layer perceptron (**MLP**)
- Convolutional neural network (**CNN**)
 - For images. Each neuron considers a cluster of pixels.
 - Downstream layers see clusters of clusters – more pixel data.
- Recurrent neural network (**RNN/LSTM**) for language
 - For language. Each neuron considers a series of words.
 - A layer learns which words to remember and which to forget.
- **Transformer** / Attention Mechanism
 - Learn which parts of pictures or sentences are important.
- Many models use combinations of all of these!

Layers for Images



AlexNet was created by graduate students Ilya Sutskever and Alex Krizhevsky with Professor **Geoffrey Hinton** at University of Toronto. It won the ImageNet contest in 2012.



The “**fully connected**” layers (FC) are exactly like our multi-layer perceptron layers.

The **convolutional** layers contain special neurons designed to summarize groups of nearby pixels.

Input layer:
pixels of the image

Output:
“anemone”

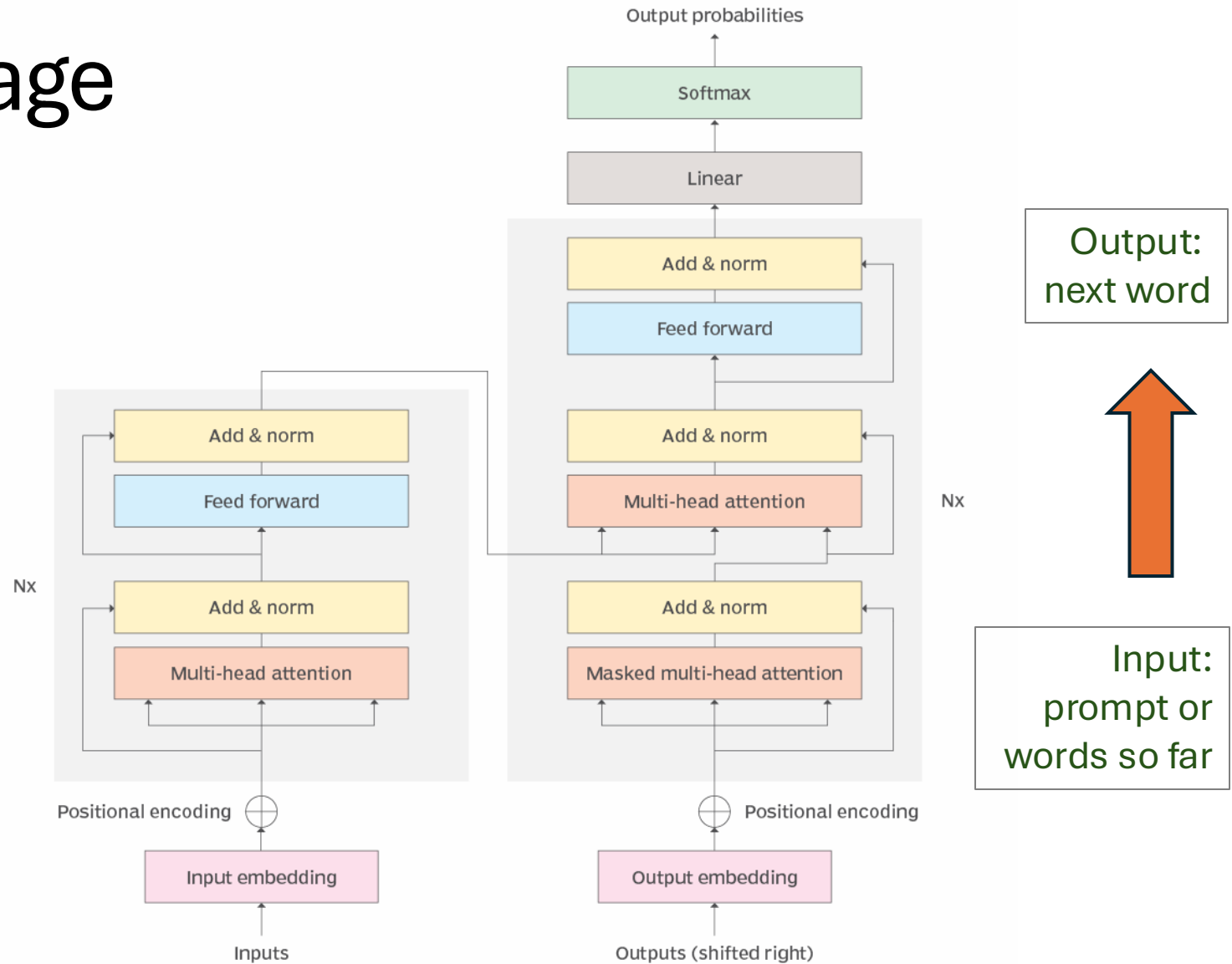


Layers for Language

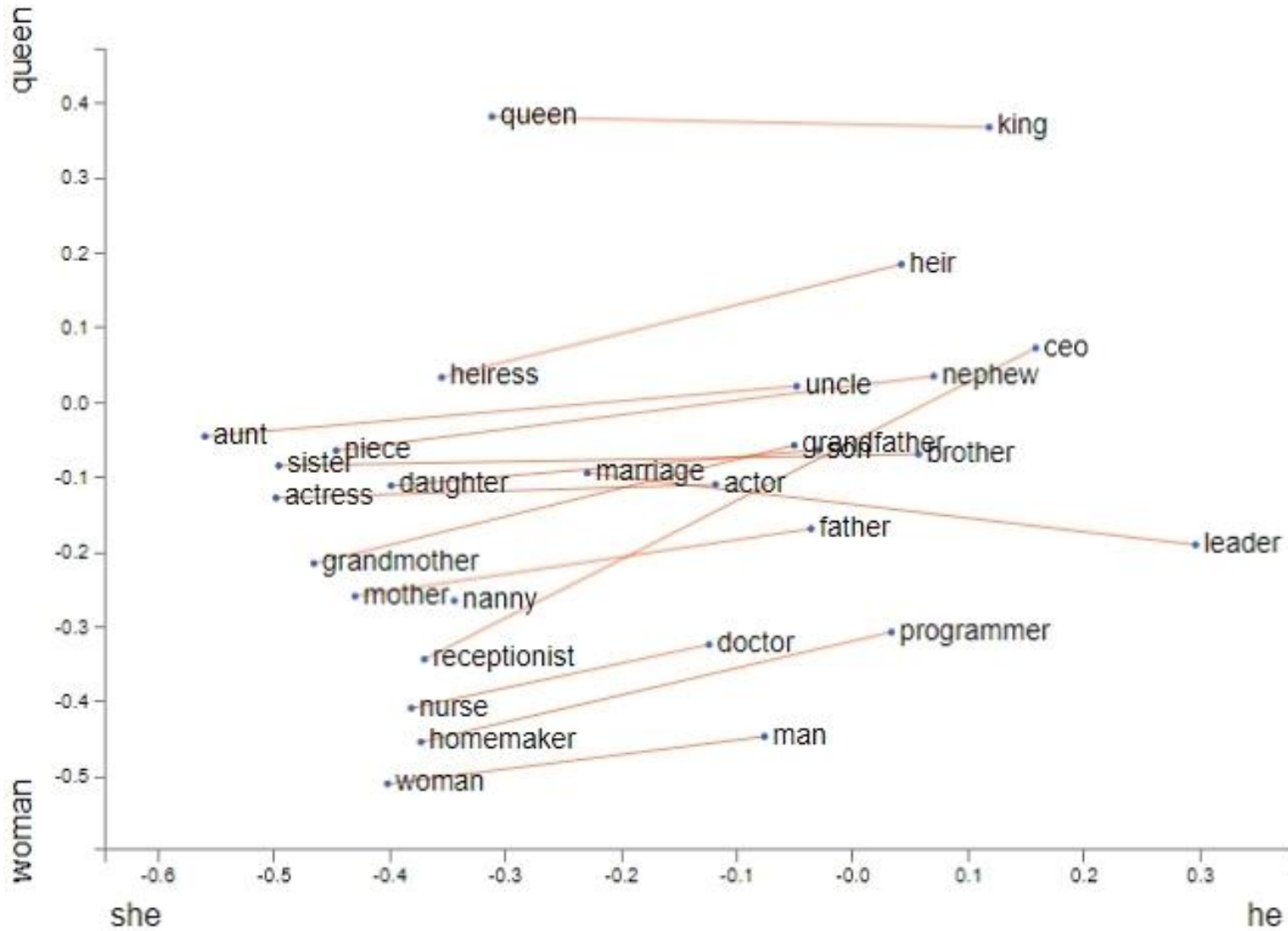
The “**feed forward**” layers are exactly like our multi-layer perceptron layers.

Transformers like GPT use “**attention**” layers to learn which words are most important.

The “**embedding**” layers convert words to numerical vectors.



Layers for Language Models



Embedding Layer

A layer found in most large language models (LLMs)

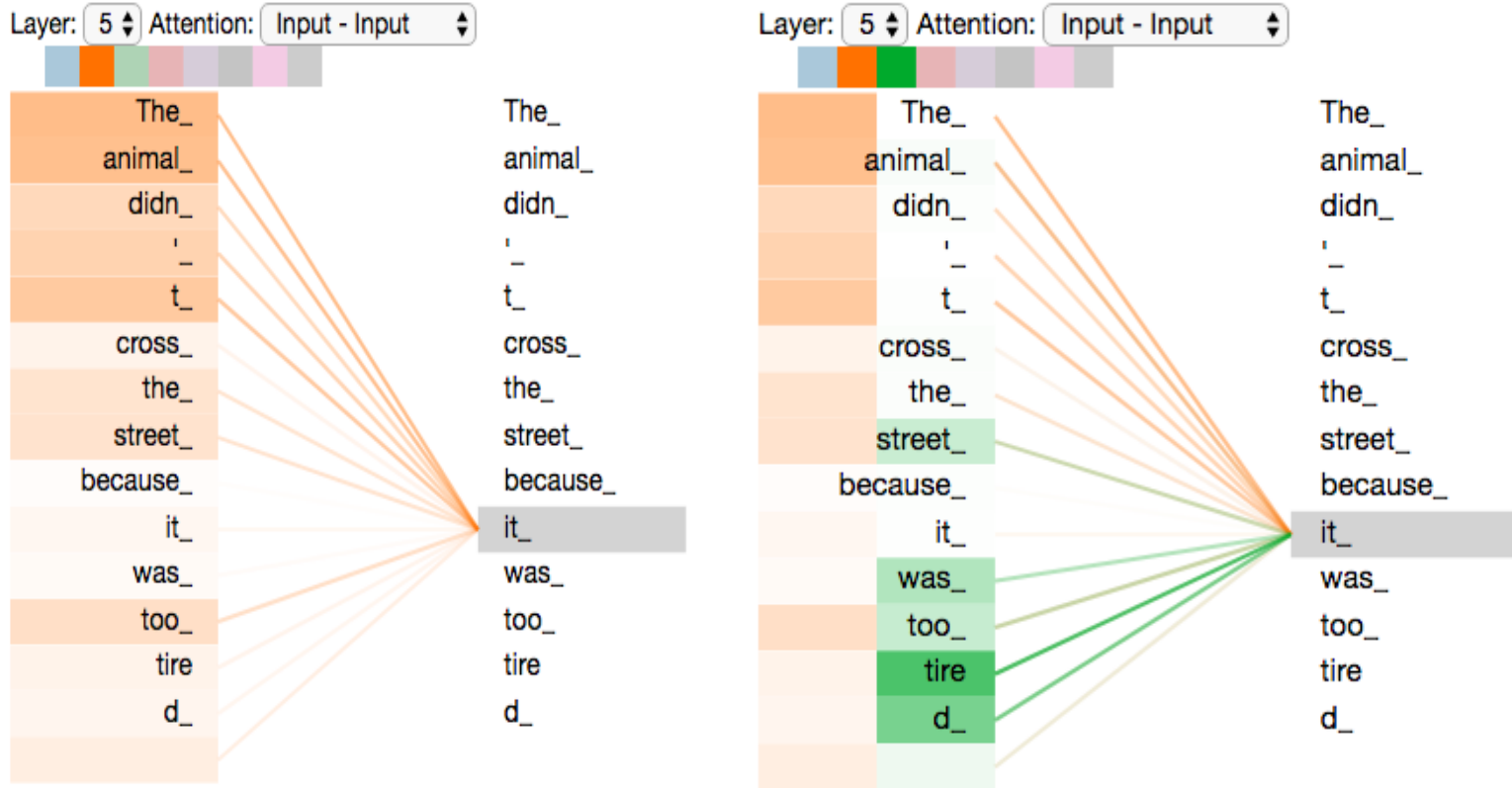
An embedding layer converts words to numbers (and numbers to words)

Each word has one position in a **multi-dimensional space**. Each position was learned from the training data.

This example uses 2 **dimensions** i.e. two numbers per word. Real embeddings have 100 or more.

Can you spot the gender bias in this example?

Layers for Language Models



Attention Layer

Any model called Transformer (the T in GPT) has attention layers. Each layer has attention heads.

Each attention head learns to focus on a different concept.

Left: one attention head is focused on "The animal" to understand "it".

Right: other attention heads concentrate on other parts of the sentence.

A neural network learns which heads to trust when.

Tutorial: MNIST Digits

- MNIST
 - A standardized data set hosted by US government (agency = “NIST”)
 - Images of handwritten digits (like the zip codes on mail envelopes)
 - Pixels are grayscale (0.0 to 1.0), displayed as white-on-black or black-on-white
 - All images are same size, centered, upright, containing exactly one digit
 - All images are labeled with the correct answer

True Label: 4



True Label: 1



True Label: 0



True Label: 7



True Label: 8



True Label: 1



True Label: 2



True Label: 7



True Label: 1



Choose a model architecture

- CNN? Overkill
 - CNN = convolutional neural network
 - Remember we looked at AlexNet
 - CNN is ideal for image classification
- MLP? Sufficient
 - MLP = multi-layer perceptron
 - Remember our feed-forward layers
 - MLP is powerful enough for this task



Choose your programming environment

- Personal Computer
 - Use Python (programming language) for access to open source code
 - Python Libraries: Keras, TensorFlow, PyTorch
 - Python **notebooks** help you develop Python bit-by-bit
 - Available from Jupyter, Zeppelin, etc.
 - Run in browsers on Unix, Mac, and Windows computers
- Google CoLab
 - Lets you run Jupyter Notebooks on the Google Cloud
 - Minimal service is free, you pay more for better service
- Hugging Face
 - Lots of code, data, and tutorials
 - Help for running in the cloud or in the browser

Further Reading

- Russell S & Norvig P (2020) Artificial Intelligence
 - The standard survey textbook by experts from Stanford University
- Poole DL & Mackworth AK (2023) Artificial Intelligence
 - Alternate textbook with simple examples in Python
- Andriy Burkov (2025) The Hundred-Page Language Models Book
 - From neural networks to LLMs, with accompanying website